CS413 · MIDTERM PROJECT

Compiling Extended LAMBDA into Pure Lambda Calculus

A compositional compiler, a call-by-value reducer, and Church-encoded results.

What the project must accomplish

Source language

Numbers, booleans, functions, application, arithmetic, conditionals, pairs, lets, and recursive functions.

Target language

Only **variables**, one-parameter **lambdas**, and **applications**.

Central constraint

The compiler translates programs compositionally; it never evaluates a source program to precompute an answer.

Evidence

Run compiled terms with a bounded beta-value reducer and decode Church-encoded results.

Two languages, one pipeline



Shared semantics

The evaluator applies left-to-right call-by-value rules to source programs and weak beta-value reduction to target terms.

Separation of concerns

Compiler, target operations, Church library, observers, demonstrations, and tests remain in separate modules.

Pure target terms and reduction

```
Term ::= Var(name)
       | Lam(name, Term)
       | App(Term, Term)

(lam x. body) v  →  body[x := v]
```

Only lambda abstractions are values. The evaluator first evaluates the function, then the argument, then contracts one beta-value step.

Weak CBV matters

- Never reduce inside a lambda body.
- A lambda completes with zero steps—even at budget zero.
- Exhaustion means another step is needed, not a proof of nontermination.
- Open target execution is rejected with its free names.

Scope safety: free variables and substitution

Capture hazard

```
[x := y] (lam y. x)
≠ (lam y. y)
= (lam fresh. y)
```

Substitution alpha-renames binders before a replacement could be captured.

Compiler naming policy

Every generated binder is deterministic: **base**, then **base_1**, **base_2**, and so on.

Names are fresh against source free names and all surrounding generated names, preserving **FV(C[e]) = FV(e)**.

Church encodings: data as behavior

Booleans

```
TRUE  = λt. λf. t
FALSE = λt. λf. f
```

Numerals

```
NUM(0) = λs. λz. z
NUM(2) = λs. λz. s (s z)
```

Pairs

```
PAIR = λa. λb. λp.
         p a b
```

The generated program contains no integer, boolean, pair, arithmetic, or recursion primitive.

Arithmetic, comparisons, and predecessor

Library combinators

SUCC, **PRED**, **ADD**, **SUB**, **MUL**, **ISZERO**, and comparisons are closed pure terms.

Subtraction iterates predecessor, so it naturally truncates at zero.

Kleene predecessor idea

```
PRED = λn. λs. λz.
  n (λg. λh. h (g s))
    (λu. z) (λu. u)
```

The continuations delay one application of *s*; the zero case returns *z*.

Compositional compilation rules

```
C[x]                 = x
C[λx. e]             = λx. C[e]
C[e₁ e₂]             = C[e₁] C[e₂]
C[let x = e₁ in e₂]  = (λx. C[e₂]) C[e₁]
C[pair(e₁,e₂)]       = PAIR C[e₁] C[e₂]
C[fst e] / C[snd e]  = FST C[e] / SND C[e]
C[fix f(x). e]       = Z (λf. λx. C[e])
```

The visitor translates each AST constructor directly and recursively; invalid literals/operators are rejected during a complete validation pass.

Call-by-value control flow needs thunks

Conditional

```
C[if c then t else f]
= C[c] (λu. C[t])
       (λv. C[f]) I
```

Only the chosen thunk receives identity **I** and is forced.

Short-circuiting

and delays its right operand unless the left is true.

or delays its right operand unless the left is false.

This preserves source behavior even when the skipped branch diverges.

Recursion under call-by-value

```
Z = λg.
 (λx. g (λv. x x v))
 (λx. g (λv. x x v))
```

Z delays self-application until a function argument arrives. Ordinary **Y** would unfold forever under call-by-value.

Translation

```
C[fix f(x). e]
= Z (λf. λx. C[e])
```

Recursive factorial and Fibonacci demonstrate that the generated target is executable, not merely printable.

Observing Church results safely

Natural numbers

```
STEP = λtail. λc. λz. c tail
BASE = λc. λz. z
```

Apply the result to STEP and BASE, then count the finite structural spine without evaluating under lambdas.

Booleans and pairs

Booleans select one of two distinct closed markers. Pairs are evaluated, projected with FST/SND, then each component is observed recursively.

Execution and observation have independent budgets and step counts.

Testing strategy

13
Unit tests

Substitution, weak CBV, budgets, library closure, encodings, scope, validation, and purity.

14
Pipeline demos

All required examples, including skipped divergence and strict divergence.

2
Recursive programs

Factorial 4 → 24 and Fibonacci 5 → 5.

Tests validate evidence—not a formal proof of semantic equivalence.

Representative results

Terminating programs

- 2 + 3 × 4 → **14**
- 2 − 5 → **0**
- if 2 ≤ 3 then 7 else 9 → **7**
- factorial 4 → **24**
- fibonacci 5 → **5**

Evaluation-order checks

- if true then 1 else LOOP → **1**
- false and LOOP → **false**
- (λx. 0) LOOP → **budget exhausted**
- LOOP → **budget exhausted**

Limitations and takeaway

Limitations

- No parser or static type checker.
- Reduction is intentionally simple and bounded.
- Observation supports assigned naturals, booleans, and finite pairs.
- Tests support confidence but do not replace proof.

Takeaway

A carefully scoped compiler can eliminate rich source primitives entirely while preserving behavior through **lexical scope**, **Church encodings**, **thunks**, and a **CBV fixed point**.

Questions?